

POLINES · ADMINISTRASI BISNIS · ABT-3D

Lembar Kerja Minggu 6 · Pertemuan 2

CRUD lewat Aplikasi Streamlit

Pemrograman Bisnis & Perancangan Sistem Administrasi · Selasa 90 menit · Lanjutan dari database kasir.db yang dibuat pada pertemuan Senin.

Nama		NIM	
Tema proyek		Nama file .db	

Petunjuk. Ikuti langkah sesuai urutan dan ketik kode persis seperti contoh. Setiap tahap punya kotak CEK: centang bila hasil Anda sama sebelum lanjut. Pakai data simulasi, jangan data pribadi pelanggan nyata. File akhir: database.py, app.py, dan lembar ini terisi.

o. Persiapan

1. Buka VS Code, pilih **File → Open Folder...**, lalu buka folder minggu-06 yang berisi kasir.db dan buat_database.py dari pertemuan Senin.
2. Salin database.py dan app.py dari folder 6.2/kode-praktik ke folder minggu-06, satu folder dengan kasir.db. Jika kasir.db belum ada, jalankan dulu python buat_database.py.
3. Pilih **Terminal → New Terminal**. Ketik python --version (Windows: py --version bila perlu). Harus tampil Python 3.
4. Periksa Streamlit: python -m streamlit version. Bila belum terpasang: python -m pip install streamlit (perlu internet). Belum bisa memasang? Pakai jalur cadangan main.py sesuai arahan dosen.

CEK ☐ Terminal berada di folder minggu-06, kasir.db ada, dan Streamlit terpasang.

BAGIAN B · CRUD lewat aplikasi Streamlit

Hari ini data dikelola lewat **halaman web Streamlit**. Tampilan dan penyimpanan dipisah ke file yang berbeda: app.py hanya memanggil fungsi di database.py. Kelas yang belum dapat memasang Streamlit memakai jalur cadangan menu terminal main.py sesuai arahan dosen; database.py tetap sama.

File	Tugas
app.py	BARU: tampilan Streamlit — formulir, tombol, pesan, dan tabel produk
database.py	BARU: menyimpan dan mengambil data dari kasir.db. Tidak berisi kode Streamlit.
logika_bisnis.py	Perhitungan dari Minggu 5 (subtotal, diskon). Tidak diubah minggu ini.

B1. Pahami database.py menit 10–20

1. Salin database.py dan app.py dari dosen ke folder minggu-06, satu folder dengan kasir.db.
2. Baca database.py. Perhatikan pola yang selalu sama: buka koneksi → try → execute → commit → finally close.
3. Perhatikan tanda tanya ?. Nilai dikirim sebagai parameter, tidak dirangkai ke teks SQL. Ini mencegah error karena tanda kutip pada nama (misalnya Toko D'Arso) dan mencegah isian jahat.

database.py (bagian yang sudah jadi)
import sqlite3

```
NAMA_DB = "kasir.db"

def buka_koneksi():
    koneksi = sqlite3.connect(NAMA_DB)
    koneksi.execute("PRAGMA foreign_keys = ON") # ulangi di setiap koneksi
    return koneksi

def tambah_produk(nama, harga, stok):
    koneksi = buka_koneksi()
    try:
        koneksi.execute(
            "INSERT INTO produk (nama, harga, stok) VALUES (?, ?, ?)",
            (nama, harga, stok),
        )
        koneksi.commit()
    finally:
        koneksi.close() # tetap ditutup walau terjadi error

def daftar_produk():
    koneksi = buka_koneksi()
    try:
        return koneksi.execute(
            "SELECT id_produk, nama, harga, stok FROM produk"
        ).fetchall()
    finally:
        koneksi.close()

def ubah_harga(id_produk, harga_baru):
    koneksi = buka_koneksi()
    try:
        cursor = koneksi.execute(
            "UPDATE produk SET harga = ? WHERE id_produk = ?",
            (harga_baru, id_produk),
        )
        koneksi.commit()
        return cursor.rowcount # 0 berarti ID tidak ditemukan
    finally:
        koneksi.close()
```

Mengapa try / finally? Bagian finally selalu dijalankan walaupun terjadi error, sehingga koneksi pasti ditutup dan file .db tidak terkunci. Catatan: with `sqlite3.connect(...)` hanya mengurus commit, tidak menutup koneksi.

B2. Lengkapi fungsi `hapus_produk` menit 20–35

Fungsi `hapus_produk(id_produk)` di bagian bawah `database.py` masih berisi `raise NotImplementedError`. Ganti isinya dengan pola seperti `ubah_harga`. Petunjuk: perintah SQL-nya `DELETE FROM produk WHERE id_produk = ?`, parameternya `(id_produk,)` (perhatikan koma), dan fungsi mengembalikan `cursor.rowcount`.

Tulis fungsi `hapus_produk` di sini, lalu salin ke `database.py`.

Awas! DELETE atau UPDATE tanpa WHERE akan mengenai SEMUA baris di tabel.

B3. Jalankan dan lengkapi `app.py` (Streamlit) menit 35–70

1. Periksa Streamlit: `python -m streamlit version`. Bila belum ada, jalankan `python -m pip install streamlit`.
2. Jalankan aplikasi: `python -m streamlit run app.py`. Browser membuka halaman **Kelola Produk**. Bila tidak terbuka, salin alamat `http://localhost:8501` dari terminal.
3. Coba tambah satu produk, lalu lihat tabel **Daftar produk** di bagian bawah. Tombol ubah harga dan hapus masih menampilkan pesan “belum dilengkapi”.
4. Untuk menghentikan aplikasi, klik terminal lalu tekan **Ctrl+C**.

```
app.py (bagian yang sudah jadi)
import sqlite3

import streamlit as st

import database

st.title("Kelola Produk")

# ----- CREATE: tambah produk (sudah jadi) -----
st.subheader("Tambah produk")
with st.form("form_tambah", clear_on_submit=True):
    nama = st.text_input("Nama produk")
    harga = st.number_input("Harga (Rp, angka bulat)", step=1, value=0)
    stok = st.number_input("Stok awal", min_value=0, step=1, value=0)
    tombol_tambah = st.form_submit_button("Simpan produk")

if tombol_tambah:
    if nama.strip() == "":
        st.error("Nama produk wajib diisi.")          # validasi Python
    else:
        try:
            database.tambah_produk(nama.strip(), harga, stok)
            st.success(f"Produk {nama.strip()} tersimpan.")
        except sqlite3.IntegrityError as e:
            st.error(f"Database menolak data: {e}")      # aturan database

# ... bagian ubah harga (Latihan E1) dan hapus (Latihan E2) ...

# ----- READ: daftar produk (sudah jadi, paling bawah) -----
st.subheader("Daftar produk")
st.dataframe(
    [
        {"ID": id_produk, "Nama": nama, "Harga": harga, "Stok": stok}
        for id_produk, nama, harga, stok in database.daftar_produk()
    ],
    hide_index=True,
```

)

Cara kerja Streamlit. Setiap kali tombol ditekan, **seluruh** `app.py` dijalankan ulang dari atas. `st.form` mengumpulkan isian sampai tombol formulir ditekan. `st.number_input(step=1)` hanya menerima angka bulat, jadi isian abc tidak bisa diketik. Tabel diletakkan paling bawah agar dibaca setelah perubahan diproses.

Latihan E1 – Ubah harga. Ganti baris `st.info(...)` pada bagian ubah harga. Panggil `database.ubah_harga(id_ubah, harga_baru):` hasil 0 → `st.warning` (ID tidak ditemukan), selainnya → `st.success`. Tangkap `sqlite3.IntegrityError` dengan `st.error`.

Tulis kode pengganti `st.info` pada bagian ubah harga di sini, lalu salin ke `app.py`.

Latihan E2 – Hapus. Ganti baris `st.info(...)` pada bagian hapus. Jika kotak konfirmasi yakin belum dicentang → `st.warning` dan **jangan** menghapus apa pun. Jika sudah dicentang → panggil `database.hapus_produk(id_hapus)`, tangani hasil 0 dan `sqlite3.IntegrityError`.

Tulis kode pengganti `st.info` pada bagian hapus di sini, lalu salin ke `app.py`.

Simpan `app.py`, lalu klik **Rerun** di pojok kanan atas halaman atau tekan **R** agar perubahan kode terbaca.

CEK ☐ Halaman Kelola Produk menampilkan tiga produk data awal; ubah harga dan hapus tidak lagi menampilkan pesan “belum dilengkapi”.

B4. Uji silang delapan skenario menit 70–85

Jalankan setiap skenario berurutan di halaman Streamlit. Setelah setiap aksi, periksa tabel **Daftar produk** di bawah halaman. Tulis hasil yang benar-benar Anda lihat, dan ambil tangkapan layar skenario 1, 4a, dan 6.

No	Skenario & langkah	Hasil yang diharapkan	Hasil saya	Sesuai?
1	Tambah: Penghapus, 2000, stok 15	Pesan hijau “Produk Penghapus tersimpan.”; muncul dengan ID baru		☐ Ya ☐ Tidak
2	Ubah harga ID 2 (Pulpen) menjadi 3500	Pesan berhasil; harga Pulpen 3500 di tabel		☐ Ya ☐ Tidak

No	Skenario & langkah	Hasil yang diharapkan	Hasil saya	Sesuai?
3	Ubah harga ID 999	Pesan kuning "ID 999 tidak ditemukan."		☐ Ya ☐ Tidak
4a	Hapus ID 1 (Buku Tulis, sudah punya transaksi), konfirmasi dicentang	Pesan merah: FOREIGN KEY constraint failed; data utuh		☐ Ya ☐ Tidak
4b	Hapus ID 3 (Map Plastik), konfirmasi dicentang	Pesan berhasil; produk lain utuh		☐ Ya ☐ Tidak
5	Tambah dengan nama kosong / hanya spasi	"Nama produk wajib diisi." (validasi Python)		☐ Ya ☐ Tidak
6	Tambah dengan harga -5	Pesan merah: CHECK constraint failed		☐ Ya ☐ Tidak
7	Hapus ID 2 tanpa mencentang konfirmasi	Pesan peringatan; tidak ada yang terhapus		☐ Ya ☐ Tidak
8	Hentikan aplikasi (Ctrl+C), jalankan lagi	Hasil skenario 1–4b tetap ada di tabel		☐ Ya ☐ Tidak

Skenario 5 ditolak oleh **Python** (lapis depan); skenario 6 ditolak oleh **database** (lapis belakang). Kolom harga sengaja tidak diberi batas minimum agar aturan CHECK dapat dibuktikan.

Satu perbaikan yang saya lakukan selama pengujian:

B5. Refleksi dan jembatan ke Minggu 7

Minggu 7 akan menambah fitur catat penjualan dan laporan di aplikasi Streamlit. Bagian mana dari pekerjaan hari ini yang tidak perlu diubah sama sekali, dan mengapa? Pada skenario 5 dan 6, lapis mana yang menolak data dan mengapa keduanya diperlukan?

Kotak bantuan: pesan error yang sering muncul

Pesan di terminal / browser	Kemungkinan penyebab dan solusi
no such table: produk	kasir.db dibuat di folder lain atau buat_database.py belum dijalankan. Pastikan terminal di folder yang sama, lalu jalankan buat_database.py.
No module named streamlit	Streamlit belum terpasang. Jalankan python -m pip install streamlit dengan perintah Python yang sama (python atau py).
'streamlit' is not recognized	Gunakan python -m streamlit run app.py, bukan streamlit run app.py.
NotImplementedError: hapus_produk()	Latihan B2 belum dikerjakan. Lengkapi hapus_produk() di database.py.

Pesan di terminal / browser	Kemungkinan penyebab dan solusi
database is locked	Ada program lain yang masih membuka kasir.db (misalnya SQLite Viewer). Tutup lalu coba lagi.
No module named 'database'	database.py tidak satu folder dengan app.py, atau nama file salah ketik (database.py.txt).
IntegrityError: ... constraint failed	Bukan bug: database menolak data yang melanggar aturan. Baca nama aturan yang disebut.
IndentationError	Jumlah spasi di awal baris tidak konsisten. Gunakan 4 spasi per tingkat.

Daftar periksa pengumpulan

☐	database.py: hapus_produk() lengkap dengan query berparameter ?
☐	PRAGMA foreign_keys = ON tetap ada di buka_koneksi()
☐	app.py berjalan dengan python -m streamlit run app.py; keempat operasi CRUD bekerja
☐	Input salah menampilkan pesan, aplikasi tidak berhenti
☐	Tabel B4 terisi hasil asli dan tangkapan layar skenario 1, 4a, 6
☐	Satu perbaikan dan refleksi B5 terisi